



- TOWER DEFENSE -
Projet de programmation fonctionnelle

Saad KIOUEH
Mohamed Taha SANDI
Adib JOUAHRI
Mohamed Dyaé CHELLAF

Encadré par M. HERBRETEAU
Département informatique S6
Mars 2023 - Mai 2023

Table des matières

1	Introduction et présentation du sujet	2
1.1	Présentation du sujet	2
1.2	Cadre du travail	3
1.3	Caractéristiques du projet	3
2	Structure du projet	4
2.1	Modules et Gestion des types	4
2.2	Manipulation des types	5
3	Combinaison des Modules	8
3.1	Structure de la boucle de jeu	8
3.2	Graphe de dépendances	10
4	Complexités et tests:	11
4.1	Complexité des fonctions:	11
4.2	Structuration des tests	12
5	Conclusion	13

1 Introduction et présentation du sujet

1.1 Présentation du sujet

Le thème du projet de programmation fonctionnelle de cette année concerne la création d'un jeu de type *Tower Defense*. Le Tower Defense est un genre de jeu vidéo dans lequel le joueur doit défendre un terrain ou une zone en utilisant des tours ou d'autres types d'acteurs. L'objectif est de protéger la zone contre les ennemis qui suivent une ou plusieurs routes prédéfinies, tout en cherchant à remplir un objectif de victoire spécifique.

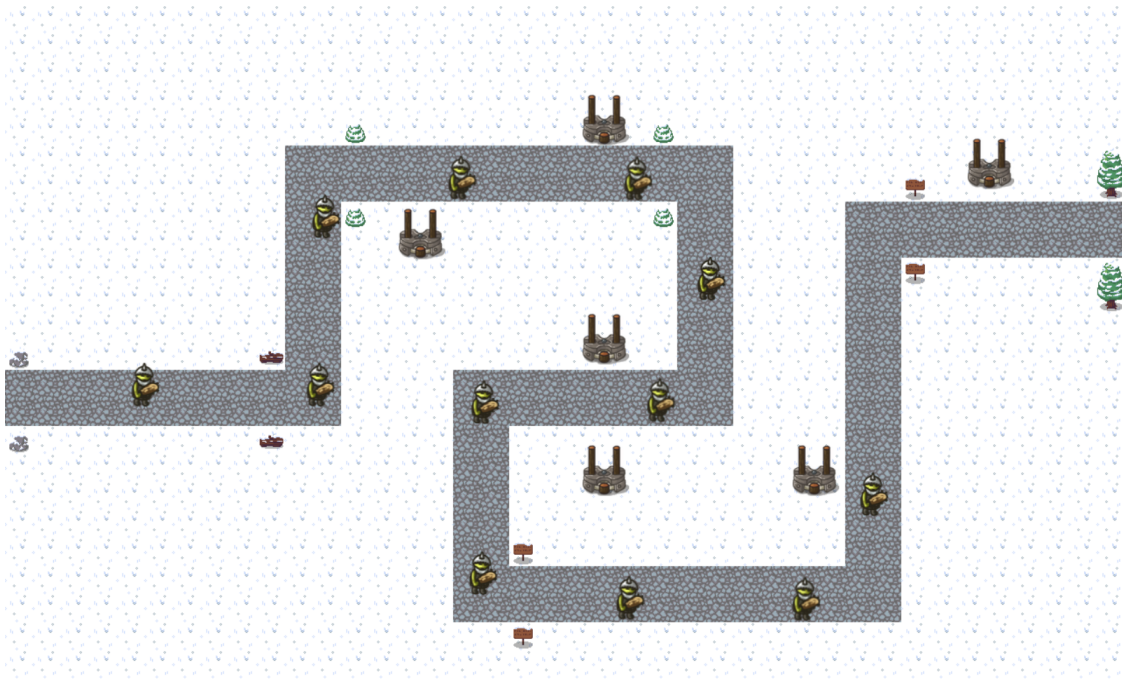


Figure 1: Exemple d'un jeu de type **Tower Defense**

Les règles du *Tower Defense* sont souvent très flexibles, offrant ainsi une grande diversité dans l'expérience de jeu. Le Tower Defense est également connu pour sa grande rejouabilité, offrant aux joueurs la possibilité de s'améliorer et de perfectionner leurs stratégies pour atteindre des scores plus élevés.

1.2 Cadre du travail

Ce projet a été réalisé en utilisant le langage *TypeScript* et suit le paradigme fonctionnel, ce qui a permis aux développeurs de mettre en œuvre plusieurs fonctions et techniques de jeu. La méthodologie de travail adoptée pour la réalisation du projet a consisté à répartir les tâches entre les quatre membres du groupe. Dans un souci d'efficacité, les membres ont travaillé principalement en sous-équipes de deux, et chacune de ces équipes était chargée de développer une partie spécifique du projet, et chaque membre de l'équipe avait la responsabilité de développer ses propres fonctions pour cette partie, ce qui a facilité le travail tout en permettant à chaque individu de s'améliorer dans les domaines de la collaboration en équipe, ainsi que dans le développement logiciel et les techniques de programmation fonctionnelle.

Les deux sous-équipes se réunissaient ensuite pour combiner leur code et discuter des choix techniques, ce qui permettait de résoudre les problèmes plus rapidement et de faire des ajustements en temps réel. Cette méthode de travail a permis de créer un environnement de collaboration efficace, où les membres du groupe ont pu s'entraider et s'encourager mutuellement à se dépasser et à améliorer leur travail en équipe.

1.3 Caractéristiques du projet

Afin de faciliter la réalisation du projet, différents aspects ont été présentés aux membres du groupe. Ces aspects peuvent être considérés comme des sous-parties du projet et doivent être priorisés en fonction de leur importance. Un algorithme pour une boucle de jeu a également été fourni, ce qui a permis de mieux structurer le développement du jeu.

Au cœur de ce projet se trouve un monde virtuel, un environnement qui peut contenir des acteurs et des cases aux caractéristiques variées. Les acteurs sont le composant le plus important de ce type de jeu, car ils constituent des entités capables de stocker leurs propres caractéristiques et comportements à chaque phase de la partie. On énumère plusieurs phases de jeu, telles que *"move"*, *"attack"*, *"heal"*, *etc.*, et des fonctions

ont été développées pour gérer les conflits qui peuvent survenir au cours de ces phases.

Il est important de souligner que cette approche par phases de jeu et de gestion des conflits est cruciale pour le bon fonctionnement de tout jeu de type *Tower Defense*, et elle a donc été mise en œuvre de manière approfondie dans le cadre de ce projet de programmation fonctionnelle.

2 Structure du projet

2.1 Modules et Gestion des types

Le projet est structuré en plusieurs fichiers, chacun représentant un module contenant des types et des fonctions spécifiques pour la manipulation de ces premiers.

Tout d’abord, le type *Position* est un type de données essentiel dans tout jeu, car une suite de positions forme un mouvement qui est une phase essentielle dans la construction d’un jeu de *Tower Defense*.

En ce qui concerne l’implémentation du monde, le choix de définir une énumération *MapElements* pour représenter les différents éléments de la carte est pertinent dans le cadre de la programmation fonctionnelle, car il permet de créer une structure de données claire et bien définie qui peut être facilement utilisée dans tout le code. En utilisant cette énumération pour représenter les différents éléments de la carte, il est facile de déterminer le type de chaque élément sans avoir à utiliser des chaînes de caractères ou d’autres types de données susceptibles de causer des problèmes (fautes de frappe). De plus, le fait d’utiliser une énumération permet de faciliter la maintenance du code, car toute modification de l’un des éléments de la carte sera facilement prise en compte dans tout le code. De plus, elle nous a permis de faciliter l’écriture des fonctions d’affichage ce qui nous a aidé à déboguer le code plus aisément. Le type *World* a également été bien conçu pour représenter les dimen-

sions et les caractéristiques de la carte de jeu. En utilisant ce type, il est possible de stocker les dimensions de la carte, les différentes tuiles qui la composent, ainsi que les positions de départ et d'arrivée des ennemis. Tout cela peut être facilement manipulé et utilisé afin de permettre aux différents acteurs de se localiser sur la carte.

Pour l'implémentation des acteurs, le choix des types est fait de façon à permettre d'intégrer différentes phases de jeu ainsi que l'interaction entre les différents acteurs. Le type *Deplacement*, qui représente un déplacement suivant x et y , est utilisé pour décrire les mouvements des acteurs dans le monde du jeu. Le fait que ce type soit immuable permet de garantir que les mouvements sont toujours cohérents et prévisibles, ce qui est important dans le cadre d'un jeu *Tower Defense*. Le type *Actor* représente un acteur du jeu avec un identifiant unique, une position dans le monde et un ensemble d'actions qui peuvent être appliquées sur/par lui. En utilisant des fonctions pour décrire les actions, cela permet de garantir que l'état de l'acteur est conservé entre chaque action et que les actions sont aussi cohérentes et prévisibles. Le type *Params* est ainsi défini pour typer les fonctions stockées dans le tableau d'actions des acteurs, puisque les actions implémentées (*i.e.* "move" et "attack") possèdent des types de retour différents.

Enfin, en ce qui concerne le type *Phase*, sa structure facilite la gestion des différentes phases au sein de notre boucle de jeu. De plus, elle permet de suivre facilement la progression d'une partie de jeu entre les différentes phases.

2.2 Manipulation des types

Tout d'abord, la fonction *initializeWorld* est mise en œuvre afin de créer un nouveau monde avec des caractéristiques qui correspondent au type *World* prédéfini. La première fonction, *generatePath*, qui implémente la route dans le plateau de jeu, est conçue de manière fonctionnelle, car elle utilise ses paramètres d'entrée sans les modifier afin de produire sa sortie.

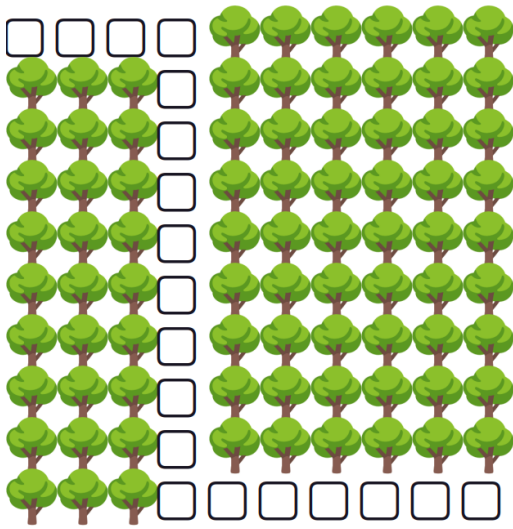


Figure 2: premier exemple d'un monde généré par *initializeWorld*

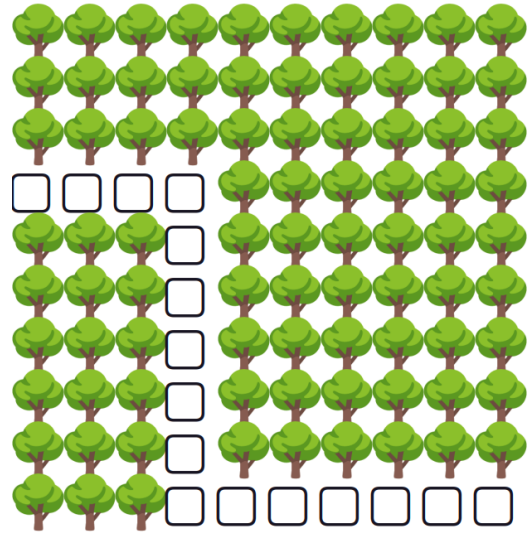


Figure 3: deuxième exemple d'un monde généré par *initializeWorld*

La fonction *getTowerPos* prend en entrée un tableau de positions *path* et un objet *World* et produit en sortie un tableau de positions valides pour placer des tours tout au long du chemin. Elle utilise également une approche fonctionnelle, faisant appel à la fonction *getNeighbor* pour obtenir les positions voisines valides et itérant sur chaque position du chemin à l'aide de la méthode *forEach*.

La fonction *initializeActors* accepte un objet *World* en entrée et produit en sortie un tableau de tableaux d'acteurs représentant les ennemis et les tours sur la carte. Cette fonction utilise une approche fonctionnelle pour initialiser les acteurs, faisant appel à la méthode *Array.from* pour créer des tableaux d'ennemis et de tours ainsi que la méthode *JSON.parse(JSON.stringify())* pour copier les attributs de l'objet d'origine plutôt que de le modifier directement.

En somme, cette partie du code fait usage de la programmation fonctionnelle pour améliorer la modularité et la réutilisabilité des fonctions. Les fonctions sont définies de manière indépendante et peuvent être combinées différemment pour générer le comportement des acteurs.

En ce qui concerne les fonctions qui permettent de régler les conflits entre les différentes phases. La fonction *findNearestRoad* est également une fonction pure qui ne modifie pas l'état global. Elle effectue simplement une recherche des voisins d'une position et renvoie cette valeur en fonction de ses entrées afin de permettre aux différents acteurs de suivre le chemin prédéfini par notre monde. De même pour la fonction *getActorsSamepos* qui utilise la méthode *JSON.stringify* pour créer une copie du tableau d'acteurs passée en paramètres et ne le modifie pas. Elle effectue ensuite une simple recherche pour trouver les acteurs pouvant se trouver dans une même position et les conserve dans un sous tableau pour ensuite donner la priorité à l'acteur le plus avancé sur le *Path*.

Dans les fonctions *resolveMovement*, *resolveAttack*, et *resolveProposals*, on peut observer l'utilisation des méthodes comme *JSON.parse* et *JSON.stringify* pour créer des copies des objets *Actor*. Pour ce qui est de la fonction *resolveMovement*, elle a été implémentée afin d'éviter les collisions entre les différents acteurs, et en ce qui concerne la fonction *resolveAttack*, cette dernière gère le conflit suivant: "*Quand deux acteurs se trouvent dans la portée d'une quelconque tour, celle-ci devra choisir l'ennemi qu'elle attaquera*". Ces fonctions ont aussi toutes des signatures de type *[World, Actor[]]*. Cela permet de spécifier clairement le type de valeur que ces fonctions retournent et facilite leur utilisation dans d'autres parties du code.

Il convient également de souligner l'utilisation de fonctions d'ordre supérieur comme *map* et *forEach* et les appliquer à des tableaux afin de produire de nouvelles valeurs. Cela permet d'améliorer la lisibilité du code.

3 Combinaison des Modules

3.1 Structure de la boucle de jeu

Avant de débiter la boucle de jeu, une condition de victoire a été préalablement établie : **"Lorsqu'un acteur ennemi atteint la dernière position de la route"**. Pour ce faire, la fonction *isGameOver* vérifie l'état du monde et des acteurs passés en paramètre sans avoir à les modifier, elle renvoie ensuite un booléen en sortie.

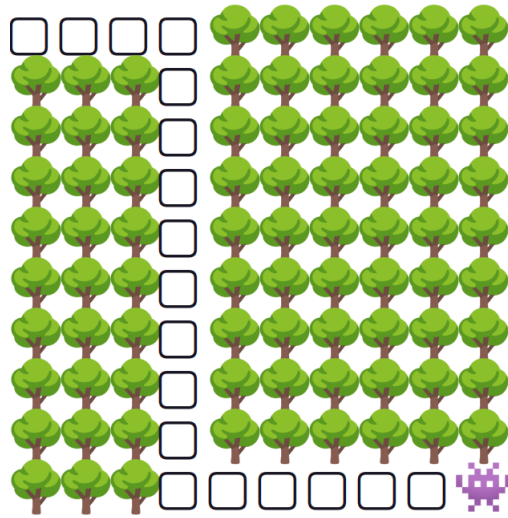


Figure 4: Affichage d'une grille ayant atteint un état de victoire

L'ensemble des fonctions implémentées tout au long du projet ont été combinées pour construire une boucle de jeu respectant le pseudo-code de la boucle de jeu proposé. En effet, celle-ci utilise la méthode *"reduce()"* pour exécuter les différentes phases du jeu. On justifie l'utilisation de *reduce* par: "Premièrement, les encadrants du projet nous encouragent à suivre une approche fonctionnelle. Deuxièmement, elle nous permet d'améliorer la lisibilité du code ce qui facilite la compréhension et le débogage du code".

De plus, la boucle utilise la fonction *resolveProposals* pour résoudre en premier lieu les conflits de mouvement et ensuite les conflits d'attaque.

Cette fonction recourt également à la fonction *printGameWorld*, qui affiche l'état du plateau de jeu après l'exécution de chaque phase, en utilisant différents caractères Unicode.

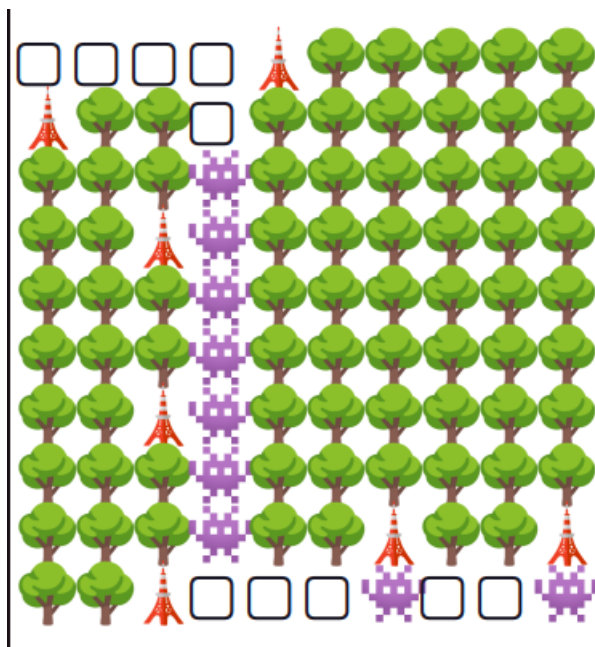


Figure 5: Exemple d'un état de la grille pendant une partie de jeu aléatoire

3.2 Graphe de dépendances

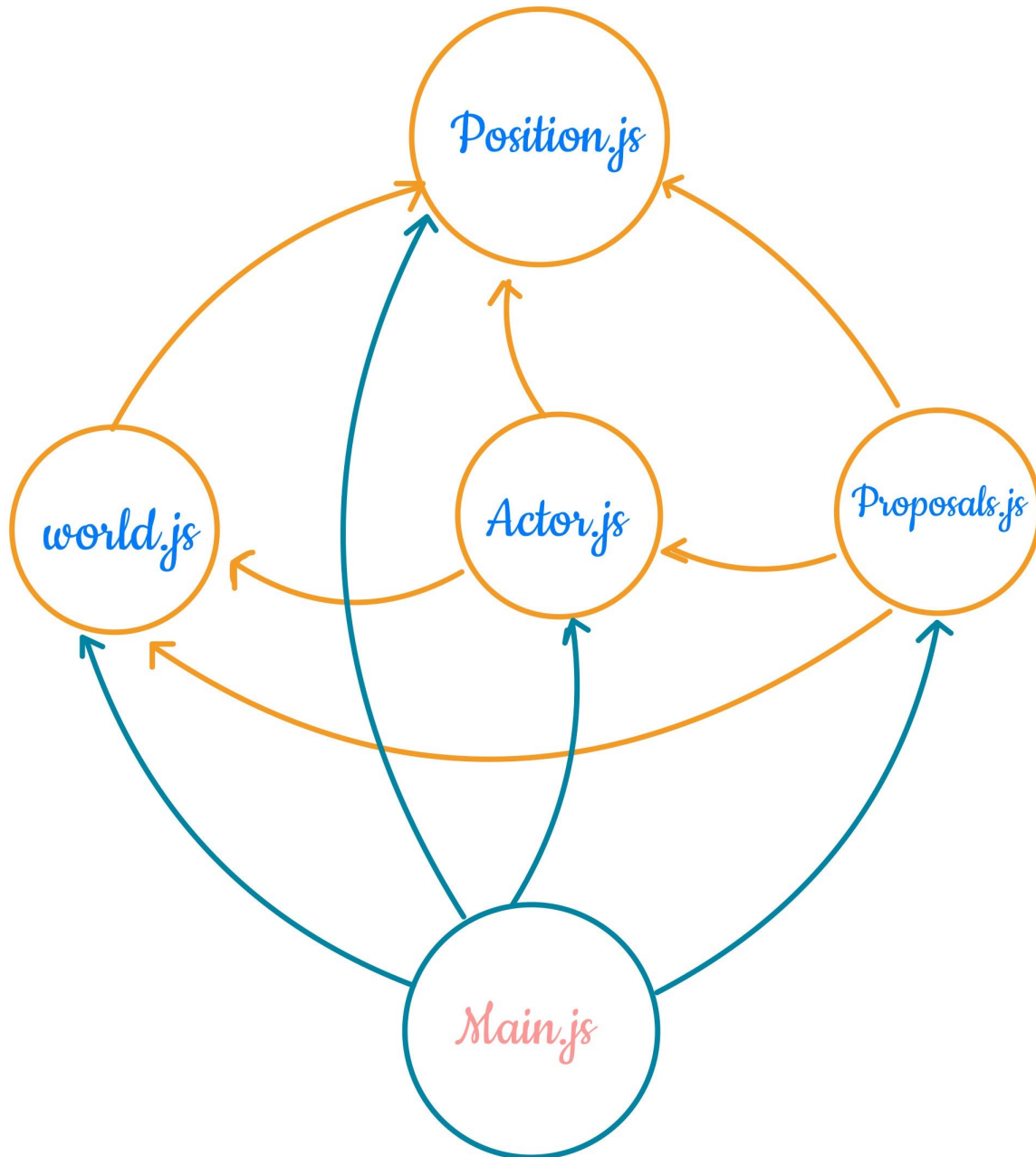


Figure 6: Graphe de dépendances

4 Complexités et tests:

4.1 Complexité des fonctions:

La fonction *generatePath* a permis de minimiser la complexité du problème en générant efficacement un chemin de manière algorithmique plutôt qu'en utilisant des méthodes coûteuses en temps comme la recherche d'un chemin à chaque appel. En utilisant une simple boucle *while* et en calculant les positions de manière systématique, la fonction génère un chemin en fonction de la position de départ et de la taille de la carte en un temps raisonnable. Ensuite, l'implémentation de *initializeWorld* parvient à générer le monde de jeu de manière très efficace, sans utiliser des boucles imbriquées ou d'opérations coûteuses, ce qui minimise la complexité du problème.

La fonction *initializeActors* utilise deux boucles, la première s'exécute en *numberEnemies* fois pour créer les ennemis, et la deuxième s'exécute en *towerpos.length* fois pour créer les tours. Cependant, puisque les deux boucles s'exécutent un nombre limité de fois, la complexité de la fonction *initializeActors* est linéaire et optimale.

En ce qui concerne les fonctions traitant les phases du jeu, la fonction *findNearestRoad* a une complexité minimale de $O(1)$. Le code examine simplement la position donnée et vérifie si les cases voisines sont de type *ROAD*. Le code ne dépend pas de la taille de la carte ou de la position donnée, donc la complexité est constante.

Pour les fonctions *getActorsSamepos* ($O(n^2)$), *resolveMovement* ($O(n^3)$) et *resolveAttack* ($O(n^2)$), bien que la complexité de la fonction ne soit pas optimale, elle est nécessaire pour effectuer les tâches requises. Cependant, il est important de noter que la copie des objets d'acteur en utilisant *JSON.parse* et *JSON.stringify* peut également prendre du temps supplémentaire, ce qui peut ralentir la fonction en fonction de la taille des objets et du tableau d'acteurs.

4.2 Structuration des tests

Afin de tester nos fonctions, nous utilisons le framework de test Jest, qui est largement utilisé pour tester des applications *TypeScript* et qui prend en charge de nombreuses fonctionnalités, telles que l'exécution de tests en parallèle, la couverture de code, le mockage de modules et de dépendances, la création de tests en mode interactif, et bien plus encore. La méthodologie utilisée suit généralement les meilleures pratiques de test, notamment en ce qui concerne l'isolation des tests et la validation des résultats attendus.

Chaque test décrit un scénario spécifique et s'assure que le résultat attendu est atteint. Les scénarios testés incluent la génération d'un chemin avec une longueur et une série de positions attendues, l'initialisation d'un monde avec des éléments de carte spécifiques et des positions de début et de fin, ainsi que des fonctions pour obtenir les coordonnées de déplacement et les positions d'acteurs spécifiques.

En utilisant une méthodologie de test rigoureuse, ces tests assurent que le code source est fonctionnel, robuste et évite les erreurs courantes. La validation des résultats attendus garantit également que les futurs changements de code ne modifient pas involontairement les fonctionnalités existantes.

En fin de compte, ces tests avaient pour objectif de nous aider à améliorer la qualité et la fiabilité de notre code, en assurant que chaque fonction et chaque module est correctement testé et validé.

5 Conclusion

Dans l'ensemble, ce projet *Tower Defense* a été une expérience enrichissante pour nous. Nous avons pu suivre un processus de développement itératif, en commençant par une version basique basée principalement sur un affichage *HTML*, et en continuant à l'améliorer et la changer jusqu'à avoir atteint notre version finale avec deux phases et deux types d'acteurs.

En ce qui concerne notre méthodologie de travail, il a été constaté que notre approche a été moins efficace à certains moments. En effet, des problèmes mineurs de communication ont été observés entre les membres du groupe. Cependant, ces difficultés ont constitué une opportunité pour améliorer notre travail en équipe et notre capacité à gérer les conflits. Nous avons ainsi pu renforcer notre collaboration et trouver des solutions conjointes aux problèmes rencontrés. En somme, cette expérience nous a permis d'améliorer nos compétences relationnelles et de développer une approche plus efficace pour travailler ensemble à l'avenir.

La programmation fonctionnelle s'est aussi montré effective pour la production d'un code plus propre et plus modulaire, grâce à l'utilisation de fonctions pures et immuables et donc facilite la gestion et l'amélioration des différentes composantes du jeu (monde, acteurs...), ainsi que les fonctions pures, qui ne modifient pas les données d'entrée, ont facilité la compréhension, la maintenance et le débogage du code (elles produisent toujours les mêmes résultats pour les mêmes entrées et sont donc prévisibles), et puisque ce type de jeu contient de nombreux acteurs et de nombreuses interactions entre eux, ces fonctions facilitent aussi la gestion de ces interactions.

Enfin, nous avons aussi testé notre jeu de Tower Defense avec différents scénarios et différentes routes et avons effectué des tests d'acceptation pour nous assurer que le jeu fonctionnait comme prévu. Nous sommes satisfaits du résultat final et nous sommes convaincus que ce projet nous a permis d'acquérir de nouvelles compétences en matière de développement de jeux et de programmation en général.